# 后方交会

## ——基于计算机视觉系统

张月莹，李佳锋，邱伟

2020/10/26

实验目的：对于三维空间中的贴画，建立局部坐标系，获取其三维坐标，在指定位置对贴画进行拍摄，获取一张相片，通过相片上的像点与三维坐标之间的关系，解算出相机在局部坐标系下的位姿，实现室内定位。

一、确定后方交会初值

1. H 矩阵计算及分解

（1） 标准化像素：

$$\begin{cases} u_s = (u - u0)/f \\ v_s = (v - v0)/f \end{cases}$$

（2） 计算三维坐标平面与 Z=0 平面之间的旋转矩阵 R_3d 与平移量 t_3d：

$$\mathrm{mm} = (M - M_C)^{\mathrm{T}}(M - M_C)$$

其中，M 为三维点坐标，是 nX3 的矩阵；Mc 为三维点重心坐标 Mcenter 组成的 nX3 的矩阵，Mc 的每一行都是 Mcenter。

$$\mathrm{mm} = \mathrm{U}\Sigma\mathrm{V}^{\mathrm{T}}$$

$$\mathrm{R}_{3\mathrm{d}} = \mathrm{V}^{\mathrm{T}}$$

$$\mathrm{t}_{3\mathrm{d}} = -R_{3d} * M_{Center}$$

上式对 mm 进行 svd 分解，V^T 为三维坐标平面与 Z=0 平面间旋转矩阵 R_3d，并求得平移量 t_3d。

（3） 将三维坐标旋转平移至 Z=0 平面得到 Z=0 平面上的坐标 Mxy_x

$$\mathrm{M}_{\mathrm{xy}} = R_{3d} * M^T + t_{3d}$$

（4） 计算标准化平面与 Z=0 平面之间的$\overline{H}$矩阵

$$\begin{bmatrix} Mxy_x & Mxy_\mathrm{y} & 1 & 0 & 0 & 0 & -Mxy_x*u_s & -Mxy_\mathrm{y}*u_s \\ 0 & 0 & 0 & Mxy_x & Mxy_\mathrm{y} & 1 & -Mxy_x*\mathrm{v}_s & -Mxy_\mathrm{y}*\mathrm{v}_s \end{bmatrix} * \begin{bmatrix} h_1 \\ h_2 \\ h_3 \\ h_4 \\ h_5 \\ h_6 \\ h_7 \\ h_8 \end{bmatrix} = \begin{bmatrix} u_s \\ v_s \end{bmatrix}$$

$$\downarrow \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad \downarrow \quad \downarrow$$

$$A \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad X \quad L$$

根据上式，其中，(Mxy_x, Mxy_y)为 Z=0 平面上的点的坐标，$(\mathrm{u_s}, v_s)$为二维点标准化像素后的坐标，一对点可列两个方程，N 对点可列 2N 个方程，使用最小二乘法求解得出$\overline{H}$矩阵，即

$$\mathrm{X} = (\mathrm{A}^{\mathrm{T}}\mathrm{A})^{-1}A^T L$$

（5） 根据$\overline{H} = \lambda[r_1, r_2, t]$矩阵组成对其进行分解，得到 Z=0 平面旋转平移至成像平面的旋转矩阵 R_2d 与平移量 t_2d

$$\mathrm{r}_1 = h_1/|h_1|$$

$$\mathrm{r}_2 = h_2/|h_2|$$

$$\mathrm{r}_3 = r_1 \times r_2 / |r_1 \times r_2|$$
$$\mathrm{R}_{2\mathrm{d}} = [r_1, r_2, r_3]$$
$$\mathrm{t}_{2\mathrm{d}} = \frac{\mathrm{h}_3}{(|h_1| + |h_2|)/2}$$

（6） 利用 R_2d 对 R_3d 进行旋转得到成像平面与三维坐标平面之间的旋转矩阵 R，同理经过旋转与平移得到平移量 t:

$$\mathrm{R} = R_{2d} * R_{3d}$$
$$\mathrm{t} = R_{2d} * t_{3d} + t_{2d}$$

（7） 将旋转矩阵 R 转换为欧拉角，将平移量 t 转换为摄影中心坐标，便于后续后方交会迭代使用

$$R = \begin{bmatrix} \cos\kappa & \sin\kappa & 0 \\ -\sin\kappa & \cos\kappa & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\varphi & 0 & -\sin\varphi \\ 0 & 1 & 0 \\ \sin\varphi & 0 & \cos\varphi \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\omega & \sin\omega \\ 0 & -\sin\omega & \cos\omega \end{bmatrix}$$
$$= \begin{bmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ a_3 & b_3 & c_3 \end{bmatrix}$$

其中，旋转矩阵的组成为：

$$\mathrm{a}_1 = \cos\varphi\cos\kappa$$
$$\mathrm{a}_2 = -\sin\kappa\cos\varphi$$
$$\mathrm{a}_3 = \sin\varphi$$
$$\mathrm{b}_1 = \cos\omega\sin\kappa + \sin\omega\sin\varphi\cos\kappa$$
$$\mathrm{b}_2 = \cos\omega\cos\kappa - \sin\omega\sin\varphi\sin\kappa$$
$$\mathrm{b}_3 = -\sin\omega\cos\varphi$$
$$\mathrm{c}_1 = \sin\omega\sin\kappa - \cos\omega\sin\varphi\cos\kappa$$
$$\mathrm{c}_2 = \sin\omega\cos\kappa + \cos\omega\sin\varphi\sin\kappa$$
$$\mathrm{c}_3 = \cos\omega\cos\varphi$$

对于摄影中心坐标可以根据下式求解：

$$\mathrm{C} = -\mathrm{R}^{-1} * t$$

二、后方交会迭代

1. 根据 P 矩阵组成：$\mathrm{P} = \mathrm{KR}[\mathrm{I}| - \mathrm{C}]$，可以得到，在计算机视觉坐标系下的共线方程为：

$$\begin{cases} u - u_0 = f\dfrac{a_1(X - X_S) + b_1(Y - Y_S) + c_1(Z - Z_S)}{a_3(X - X_S) + b_3(Y - Y_S) + c_3(Z - Z_S)} \\ v - v_0 = f\dfrac{a_2(X - X_S) + b_2(Y - Y_S) + c_2(Z - Z_S)}{a_3(X - X_S) + b_3(Y - Y_S) + c_3(Z - Z_S)} \end{cases}$$

为了便于后续求导，记：

$$\begin{cases}\bar{X} = a_1(X - X_S) + b_1(Y - Y_S) + c_1(Z - Z_S) \\ \bar{Y} = a_2(X - X_S) + b_2(Y - Y_S) + c_2(Z - Z_S) \\ \bar{Z} = a_3(X - X_S) + b_3(Y - Y_S) + c_3(Z - Z_S)\end{cases}$$

2. 对共线方程进行线性化，根据非齐次线性方程组解法对六个外方位元素的改正值进行迭代求解。
3. 共线方程偏导数：

$$\frac{\partial u}{\partial X_S} = \frac{1}{\bar{Z}} * [a_3(u - u_0) - a_1 f] \qquad \frac{\partial v}{\partial X_S} = \frac{1}{\bar{Z}} * [a_3(v - v_0) - a_2 f]$$

$$\frac{\partial u}{\partial Y_S} = \frac{1}{\bar{Z}} * [b_3(u - u_0) - b_1 f] \qquad \frac{\partial v}{\partial Y_S} = \frac{1}{\bar{Z}} * [b_3(v - v_0) - b_2 f]$$

$$\frac{\partial u}{\partial Z_S} = \frac{1}{\bar{Z}} * [c_3(u - u_0) - c_1 f] \qquad \frac{\partial v}{\partial Z_S} = \frac{1}{\bar{Z}} * [c_3(v - v_0) - c_2 f]$$

$$\frac{\partial u}{\partial \omega} = \frac{1}{\bar{Z}} * [f * (b_1(Z - Z_S) - c_1(Y - Y_S)) - (u - u_0) * (b_3(Z - Z_S) - c_3(Y - Y_S))]$$

$$\frac{\partial v}{\partial \omega} = \frac{1}{\bar{Z}} * [f * (b_2(Z - Z_S) - c_2(Y - Y_S)) - (v - v_0) * (b_3(Z - Z_S) - c_3(Y - Y_S))]$$

$$\frac{\partial u}{\partial \varphi} = \frac{f}{\bar{Z}} * [-sin\varphi cos\kappa(X - X_S) + sin\omega cos\varphi cos\kappa(Y - Y_S) - cos\omega cos\varphi cos\kappa(Z - Z_S)]$$

$$-\frac{u - u_0}{\bar{Z}} * [\cos\varphi(X - X_S) + \sin\omega\sin\varphi(Y - Y_S) - \cos\omega\sin\varphi(Z - Z_S)]$$

$$\frac{\partial v}{\partial \varphi} = \frac{f}{\bar{Z}} * [sin\varphi sin\kappa(X - X_S) - sin\omega cos\varphi sin\kappa(Y - Y_S) + cos\omega cos\varphi sin\kappa(Z - Z_S)]$$

$$-\frac{v - v_0}{\bar{Z}} * [\cos\varphi(X - X_S) + \sin\omega\sin\varphi(Y - Y_S) - \cos\omega\sin\varphi(Z - Z_S)]$$

$$\frac{\partial u}{\partial \kappa} = f * \frac{\bar{Y}}{\bar{Z}} = v - v_0$$

$$\frac{\partial v}{\partial \kappa} = -f * \frac{\bar{X}}{\bar{Z}} = -(u - u_0)$$

三、详细代码

```
clc;
clear;

format long g
x0=1487.240000;y0=1973.510000;
f=(3029.350000+3032.070000)/2;
k1=0.0488249;k2=-0.168801;k3=0.194688;p1=-0.000276;p2=0.044539;

% 读取数据
fid = fopen('1.txt');
C = textscan(fid, '%f %f %f %f %f','HeaderLines',8 ,'Delimiter',
',');
fclose(fid);
m = [C{1},C{2}];
M = [C{3},C{4},C{5}];
```

```
[row,col]=size(M);

%step1 标准化像素
un_m=zeros(row,2);
for i=1:row
%
[un_x,un_y]=removeDistortion(m(i,1),m(i,2),x0,y0,f,k1,k2,k3,p1,p2);
    un_m(i,1)=(m(i,1)-x0)/f;
    un_m(i,2)=(m(i,2)-y0)/f;
end

%step2 对三维空间点坐标进行重心化并对其进行svd分解
Mc=sum(M)/row;
MM=zeros(row,3);
for i=1:row
    MM(i,:)=M(i,:)-Mc;
end
mm=MM'*MM;
[U,W,V]=svd(mm);

%step3 将svd分解的结果作为子旋转矩阵，并将三维空间坐标旋转平移至xoy平面上
R_temp=V';
T_temp=-R_temp*Mc';
Mxy=zeros(row,2);
for i=1:row
    XYm=zeros(row,2);
    XYm(i,1)=R_temp(1,:)*M(i,:)'+T_temp(1);
    XYm(i,2)=R_temp(2,:)*M(i,:)'+T_temp(2);
    Mxy(i,:)=[XYm(i,1),XYm(i,2)];
end

%计算H-矩阵
H = calH(Mxy,un_m)
h1=H(:,1);h2=H(:,2);t=H(:,3);

%分解H-矩阵获得相机位姿
h1_norm=sqrt(h1(1)*h1(1)+h1(2)*h1(2)+h1(3)*h1(3));
h2_norm=sqrt(h2(1)*h2(1)+h2(2)*h2(2)+h2(3)*h2(3));
r1=h1/h1_norm;
r2=h2/h2_norm;
t=t/((h1_norm+h2_norm)/2);
r3=cross(r1,r2);
r3=r3/norm(r3);
```

```
R=[r1,r2,r3]
T=R*T_temp+t
R_last=R*R_temp
T_last=-inv(R_last)*T
 [r1,r2,r3] = dcm2angle(R_last,'XYZ');

sum=0;
diffEO = 1;
EO=[T_last(1),T_last(2),T_last(3),r1,r2,r3]';
xi=zeros(row,3);
XYZm=zeros(row,3);
xy_temp=zeros(row,2);
A=zeros(row,6);
L=zeros(row,1);
%% omega phi kappa
while diffEO > 1e-5 && sum < 500

    omega = EO(4);
    phi = EO(5);
    kappa = EO(6);
    C1 = EO(1:3);

    a1=cos(phi)*cos(kappa);
    a2=-cos(phi)*sin(kappa);
    a3=sin(phi);
    b1=sin(omega)*sin(phi)*cos(kappa)+cos(omega)*sin(kappa);
    b2=-sin(omega)*sin(phi)*sin(kappa)+cos(omega)*cos(kappa);
    b3=-sin(omega)*cos(phi);
    c1=-cos(omega)*sin(phi)*cos(kappa)+sin(omega)*sin(kappa);
    c2=cos(omega)*sin(phi)*sin(kappa)+sin(omega)*cos(kappa);
    c3=cos(omega)*cos(phi);
    R=[a1 b1 c1;
       a2 b2 c2;
       a3 b3 c3];

   cosw = cos(omega);
   sinw = sin(omega);
   cosp = cos(phi);
   sinp = sin(phi);
   cosk = cos(kappa);
   sink = sin(kappa);

    for i=1:size(m,1)
        xi(i,:) = M(i,1:3)'- C1; %XA-XS
```

```
        XYZm(i,:) = R * xi(i,:)';

        xy_temp(i,1) = f * XYZm(i,1) / XYZm(i,3) + x0;
        xy_temp(i,2) = f * XYZm(i,2)/XYZm(i,3) + y0;
        xyp = xy_temp(i,:);

        X_ = XYZm(i,1);
        Y_ = XYZm(i,2);
        Z_ = XYZm(i,3);

        x_x0 = xyp(1) - x0;
        y_y0 = xyp(2) - y0;

        X_XS = xi(i,1);
        Y_YS = xi(i,2);
        Z_ZS = xi(i,3);

        dx_XS = 1 / Z_ * (-f * a1 + a3 * x_x0);
        dx_YS = 1 / Z_ * (-f * b1 + b3 * x_x0);
        dx_ZS = 1 / Z_ * (-f * c1 + c3 * x_x0);
        dx_omega = f / Z_ * (b1 * Z_ZS- c1 * Y_YS) - x_x0 / Z_ * (b3 *
Z_ZS - c3 * Y_YS);
        temp_one = f / Z_ * (-sinp*cosk * X_XS + sinw*cosp*cosk * Y_YS
- cosw*cosp*cosk * Z_ZS);
        temp_two = -x_x0 / Z_ * (cosp * X_XS + sinw*sinp * Y_YS -
cosw*sinp * Z_ZS);
        dx_phi =  temp_one + temp_two;
        dx_kappa = f / Z_ * Y_;
%         dx_kappa = y_y0 ;

        dy_XS = 1 / Z_ * (-f * a2 + a3 * y_y0);
        dy_YS = 1 / Z_ * (-f * b2 + b3 * y_y0);
        dy_ZS = 1 / Z_ * (-f * c2 + c3 * y_y0);
        dy_omega = f / Z_ * (b2 * Z_ZS- c2 * Y_YS) - y_y0 / Z_ * (b3 *
Z_ZS - c3 * Y_YS);
        temp_one = f / Z_ * (sink*sinp * X_XS - sinw*sink*cosp * Y_YS
+ cosw*sink*cosp * Z_ZS);
        temp_two = -y_y0 / Z_ * (cosp * X_XS + sinw*sinp * Y_YS -
cosw*sinp * Z_ZS);
        dy_phi =  temp_one + temp_two;
        dy_kappa = -f / Z_ * X_;
%         dy_kappa = -x_x0 ;

        A(2*i-1,:)=[dx_XS dx_YS dx_ZS dx_omega dx_phi dx_kappa];
```

```
        A(2*i  ,:)=[dy_XS dy_YS dy_ZS dy_omega dy_phi dy_kappa];
        L(2*i-1)=m(i,1)-xyp(1);
        L(2*i)=m(i,2)-xyp(2);
    end
    dEO = inv(A'*A) * (A'*L);
    diffEO = max(dEO);
    EO = EO + dEO;
    sum = sum + 1;
end
```